

Kubernetes Assignment

1. Application Overview

This project demonstrates the deployment and management of a frontend application using Kubernetes. The application is designed with two distinct versions to showcase version control and rolling updates within a containerized environment.

The frontend is implemented using basic HTML and is intentionally kept simple, as permitted by the assignment guidelines. Two versions of the application were created:

- **Version 1 (v1):** Displays a system dashboard interface indicating a stable system state.
- **Version 2 (v2):** Displays an updated version of the dashboard with visible UI changes and an updated system status.

The difference between the two versions is clearly visible in the browser, fulfilling the requirement to demonstrate version changes effectively.

2. Docker Image Creation

To containerize the application, Docker was used to build two separate images corresponding to the two versions of the frontend.

The first image (v1) contains the initial version of the application, while the second image (v2) contains the updated version with visible UI modifications. Both images were built locally using Docker and then pushed to Docker Hub to make them accessible for Kubernetes deployment.

The following commands were used:

Bash

```
docker build -t lokeshmpt/k8s-app:v1 .  
docker build -t lokeshmpt/k8s-app:v2 .  
docker push lokeshmpt/k8s-app:v1  
docker push lokeshmpt/k8s-app:v2
```

3. Kubernetes YAML Configuration

Four Kubernetes configuration files were created to demonstrate different concepts of container orchestration.

- **pod.yml:** Defines a single Pod running the application container, representing the most basic unit in Kubernetes.
- **replicaset.yml:** Ensures that multiple instances of the application are running simultaneously, providing scalability and fault tolerance.

- **deployment.yml:** The most important component, as it manages the application lifecycle. It includes a rolling update strategy and is configured with three probes:
 - **Liveness Probe:** Checks if the container is running.
 - **Readiness Probe:** Checks if the container is ready to serve traffic.
 - **Startup Probe:** Ensures proper initialization before checks begin.
- **service.yml:** Exposes the application using a NodePort service, allowing it to be accessed through a web browser.

4. Deployment Process

The Kubernetes resources were created using the following commands:

Bash

```
kubectl apply -f pod.yml
kubectl apply -f replicaset.yml
kubectl apply -f deployment.yml
kubectl apply -f service.yml
```

Once deployed, the status of the pods was monitored continuously using:

Bash

```
kubectl get pods --watch
```

To access the application in the browser, the following command was used:

Bash

```
minikube service app-service --url
```

This provided a URL through which the application could be viewed.

5. Rolling Update Demonstration

Initially, the application was deployed using the Version 1 (v1) Docker image. The application was verified in the browser to confirm that Version 1 was running successfully.

To demonstrate Kubernetes rolling updates, the application image was updated to Version 2 (v2) using the following command:

Bash

```
kubectl set image deployment/app-deployment app-container=lokeshmpt/k8s-app:v2
```

During this process, Kubernetes gradually terminated the existing v1 pods and replaced them with new v2 pods. This transition occurred without downtime, ensuring continuous availability

of the application.

The update process was observed using:

Bash

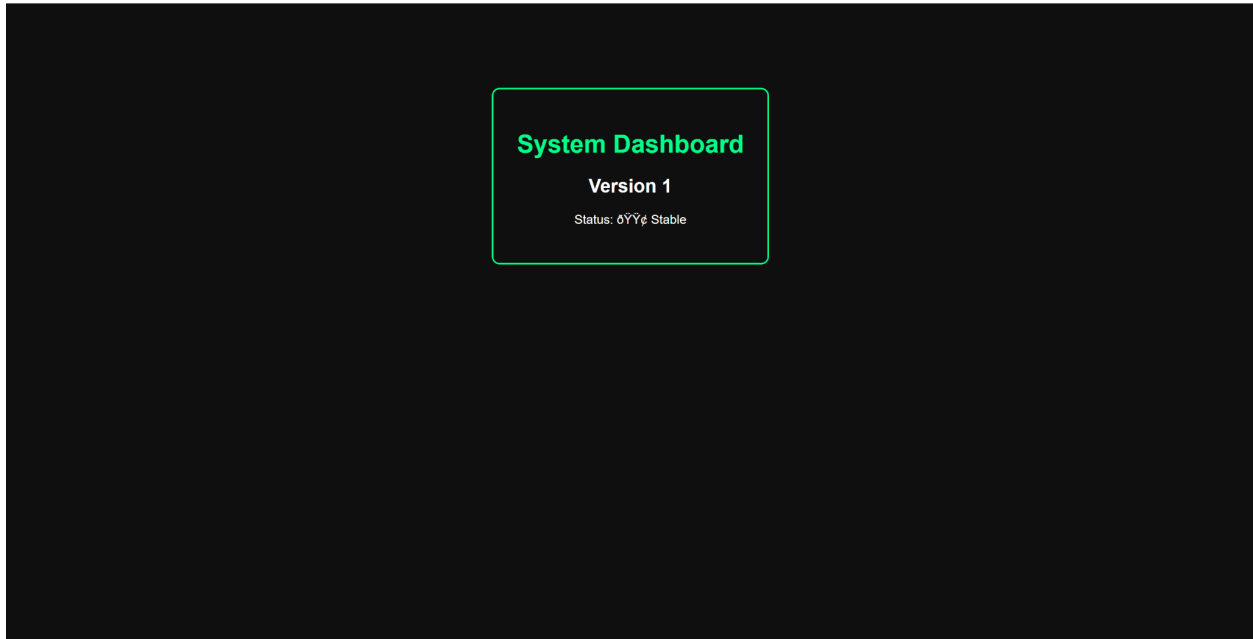
```
kubect! get pods --watch
```

After completion, all running pods were based on the v2 image, and the browser displayed the updated Version 2 interface.

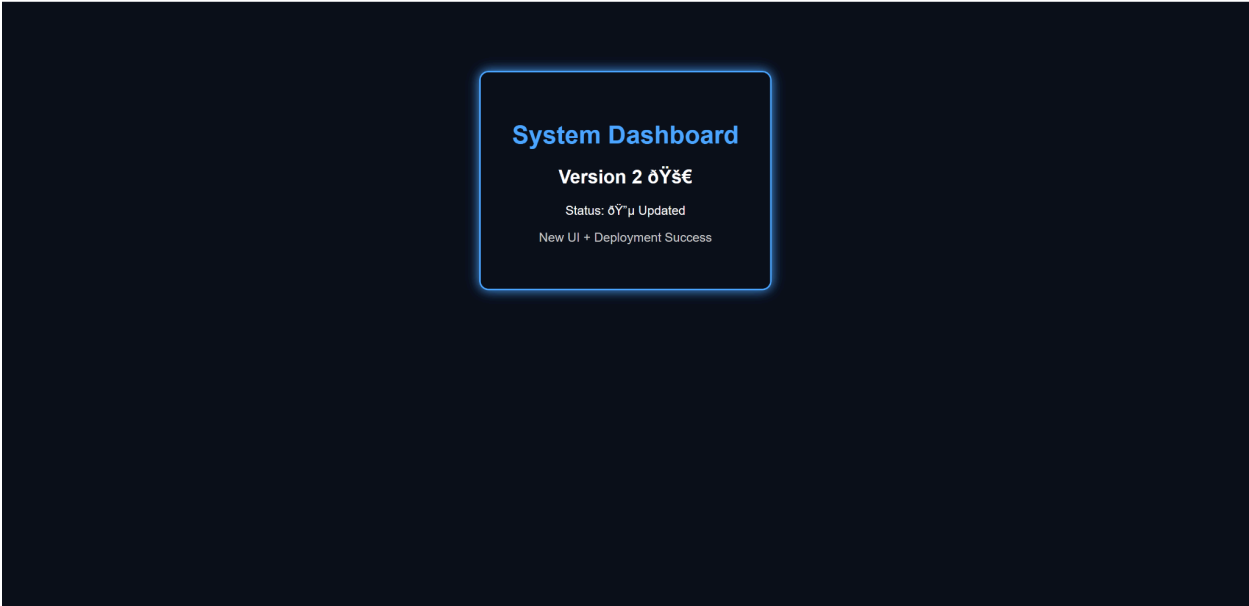
6. Screenshots

The following screenshots are included as proof of execution:

1. Application output showing Version 1



2. Application output showing Version 2



3. Docker images list

```
PS D:\iitm_projects\Day 6-20260427T105803Z-3-001\Day 6\Assignments\kubernetes_final_assignment\kubernetes> docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
cassandra:4.1	bfc28ce8118c	504MB	152MB	U
conversation_management-backend:latest	da2707365d89	423MB	139MB	U
conversation_management-frontend:latest	5396c1b612a5	100MB	27.5MB	U
gcr.io/k8s-minikube/kicbase:v0.0.50	b97074569ae9	1.94GB	545MB	
ger-io/k8s-minikube/kicbase@sha256:eb4fec00e8ad70ad78e6436f195cc429825ffb85f95afc5dbd9deb576f3e93	eb4fec00e8ad	1.94GB	545MB	U
lokeshmt/k8s-app:v1	9cf8e75b3f09	92.6MB	26MB	
lokeshmt/k8s-app:v2	cc05ee189284	92.6MB	26MB	
postgres:15	29342cb52157	633MB	164MB	U

```
PS D:\iitm_projects\Day 6-20260427T105803Z-3-001\Day 6\Assignments\kubernetes_final_assignment\kubernetes>
```

4. Running pods in Kubernetes

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\iitm_projects\Day 6-20260427T105803Z-3-001\Day 6\Assignments\kubernetes_final_assignment\kubernetes> kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
app-deployment-5dcc584d68-n9p8d	1/1	Running	0	29m
app-deployment-5dcc584d68-wmsqv	1/1	Running	0	29m
app-pod	1/1	Running	0	32m
app-rs-srlnt	1/1	Running	0	31m
app-rs-628qh	1/1	Running	0	31m

```
PS D:\iitm_projects\Day 6-20260427T105803Z-3-001\Day 6\Assignments\kubernetes_final_assignment\kubernetes>
```

5. Rolling update process

```
PS D:\iitm_projects\Day 6-20260427T105803Z-3-001\Day 6\Assignments\kubernetes_final_assignment> kubectl get pods --watch
```

NAME	READY	STATUS	RESTARTS	AGE
app-deployment-dc7bf88c9-h2bnb	1/1	Running	0	52s
app-deployment-dc7bf88c9-k6csp	1/1	Running	0	52s
app-pod	1/1	Running	0	84s
app-rs-5rlnt	1/1	Running	0	63s
app-rs-628qh	1/1	Running	0	63s
app-deployment-5dcc584d68-n9p8d	0/1	Pending	0	0s
app-deployment-5dcc584d68-n9p8d	0/1	Pending	0	0s
app-deployment-dc7bf88c9-k6csp	1/1	Terminating	0	110s
app-deployment-5dcc584d68-n9p8d	0/1	ContainerCreating	0	1s
app-deployment-5dcc584d68-wmsqv	0/1	Pending	0	0s
app-deployment-dc7bf88c9-k6csp	1/1	Terminating	0	110s
app-deployment-5dcc584d68-wmsqv	0/1	Pending	0	0s
app-deployment-5dcc584d68-wmsqv	0/1	ContainerCreating	0	0s
app-deployment-dc7bf88c9-k6csp	0/1	Completed	0	110s
app-deployment-dc7bf88c9-k6csp	0/1	Completed	0	111s
app-deployment-dc7bf88c9-k6csp	0/1	Completed	0	111s
app-deployment-5dcc584d68-n9p8d	0/1	Running	0	3s
app-deployment-5dcc584d68-wmsqv	0/1	Running	0	2s
app-deployment-5dcc584d68-n9p8d	0/1	Running	0	6s
app-deployment-5dcc584d68-wmsqv	0/1	Running	0	5s
app-deployment-5dcc584d68-n9p8d	1/1	Running	0	7s
app-deployment-5dcc584d68-wmsqv	1/1	Running	0	6s
app-deployment-dc7bf88c9-h2bnb	1/1	Terminating	0	116s
app-deployment-dc7bf88c9-h2bnb	1/1	Terminating	0	116s
app-deployment-dc7bf88c9-h2bnb	0/1	Completed	0	116s
app-deployment-dc7bf88c9-h2bnb	0/1	Completed	0	117s
app-deployment-dc7bf88c9-h2bnb	0/1	Completed	0	117s

6. Service access in browser

```
PS D:\iitm_projects\Day 6-20260427T105803Z-3-001\Day 6\Assignments\kubernetes_final_assignment\kubernetes> minikube service app-service --url
http://127.0.0.1:61048
! Because you are using a Docker driver on windows, the terminal needs to be open to run it.
```

7. Conclusion

This assignment successfully demonstrates the deployment and management of an application using Kubernetes. It covers the complete workflow, including containerization, scaling, deployment, service exposure, and rolling updates.

Through this implementation, the key concepts of Kubernetes such as Pods, ReplicaSets, Deployments, Services, and health probes have been practically understood and applied.